Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: Aguilar Ardila Gabriel Esteban **Fecha:** 22/02/2026 **Asignatura:** Estructura de Computadores

Enlace del repositorio en GitHub: https://github.com/GaboAguilar-cloud/Gabriel-Aguilar-estructura-computadores-act01

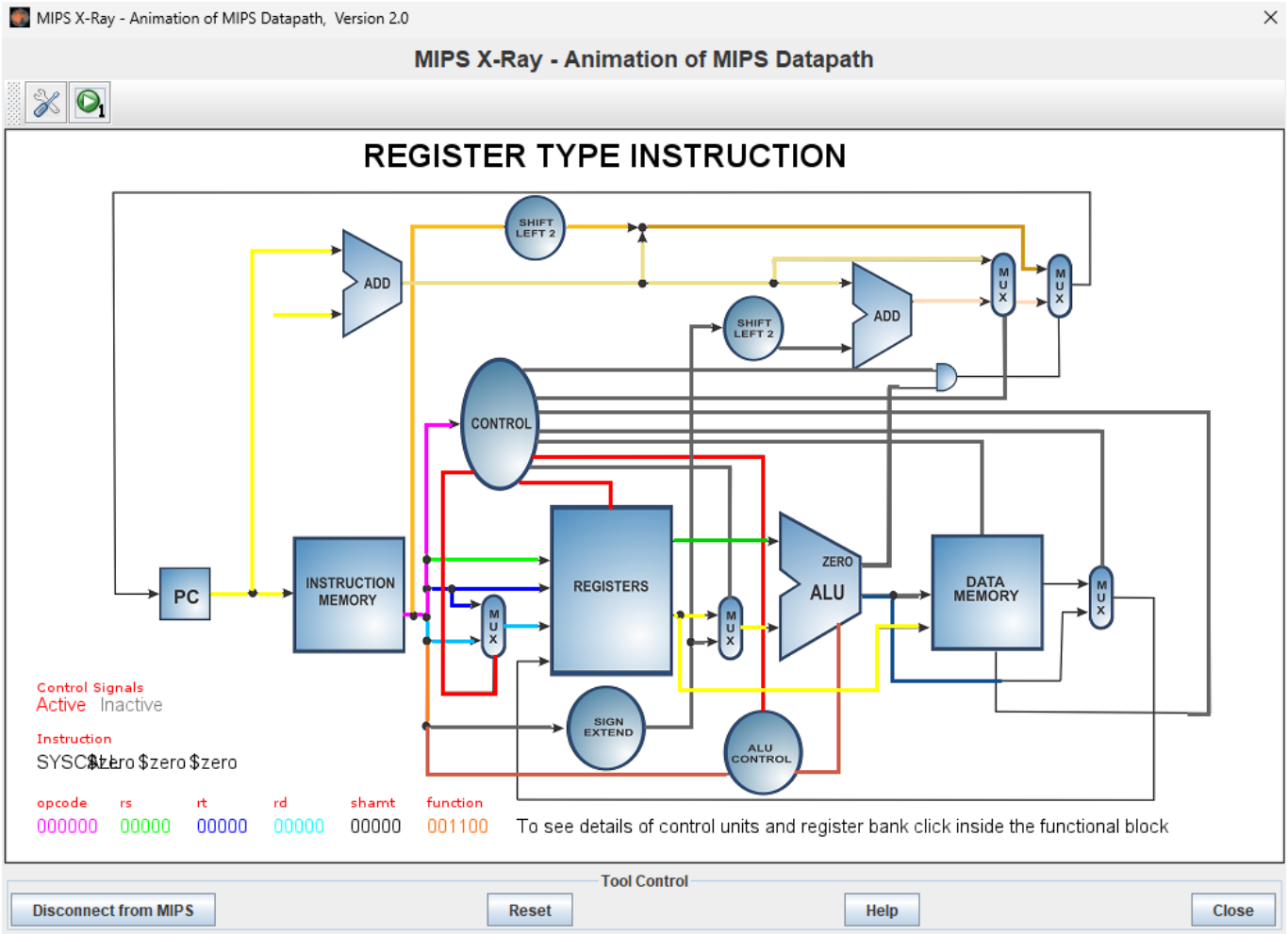
---

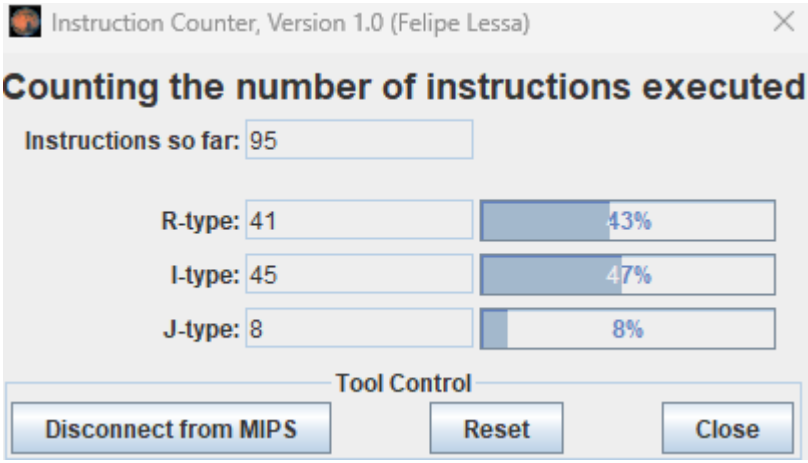
1. Análisis del Código Base

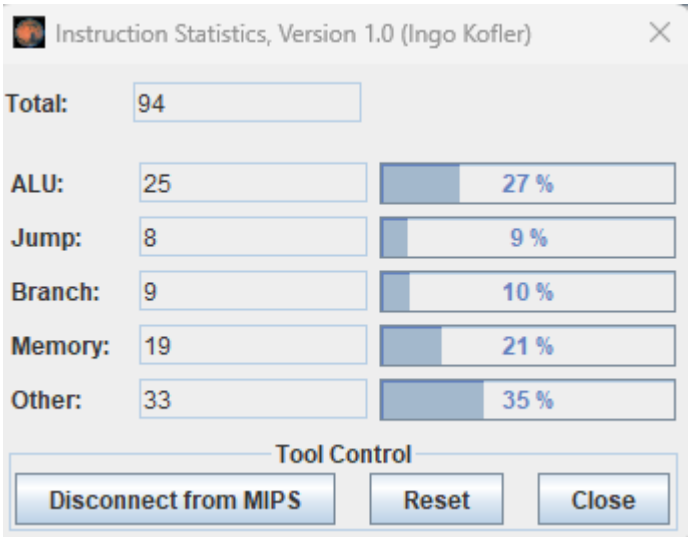
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

- **MIPS X-Ray** (Ventana con el Datapath animado).
- **Instruction Counter** (Contador de instrucciones totales).
- **Instruction Statistics** (Desglose por tipo de instrucción).


MIPS X-Ray - Animation of MIPS Datapath, Version 2.0
MIPS X-Ray - Animation of MIPS Datapath
REGISTER TYPE INSTRUCTION
SHIFT LEFT 2
ADD
ADD
SHIFT LEFT 2
MUX
MUX
CONTROL
PC
INSTRUCTION MEMORY
REGISTERS
ZERO
ALU
DATA MEMORY
MUX
MUX
MUX
SIGN EXTEND
ALU CONTROL
Control Signals
Active Inactive
Instruction
SYSCALL $zero $zero $zero
opcode rs rt rd shamt function
000000 00000 00000 00000 00000 001100
To see details of control units and register bank click inside the functional block
Tool Control
Disconnect from MIPS
Reset
Help
Close


Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 95
R-type: 41
43%
I-type: 45
47%
J-type: 8
8%
Tool Control
Disconnect from MIPS
Reset
Close


Instruction Statistics, Version 1.0 (Ingo Kofler)
Total: 94
ALU: 25
27 %
Jump: 8
9 %
Branch: 9
10 %
Memory: 19
21 %
Other: 33
35 %
Tool Control
Disconnect from MIPS
Reset
Close

1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo (Load-Use, etc.)	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	Load-Use y RAW	4 (2 de Load-Use y 2 del mul-addu)

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

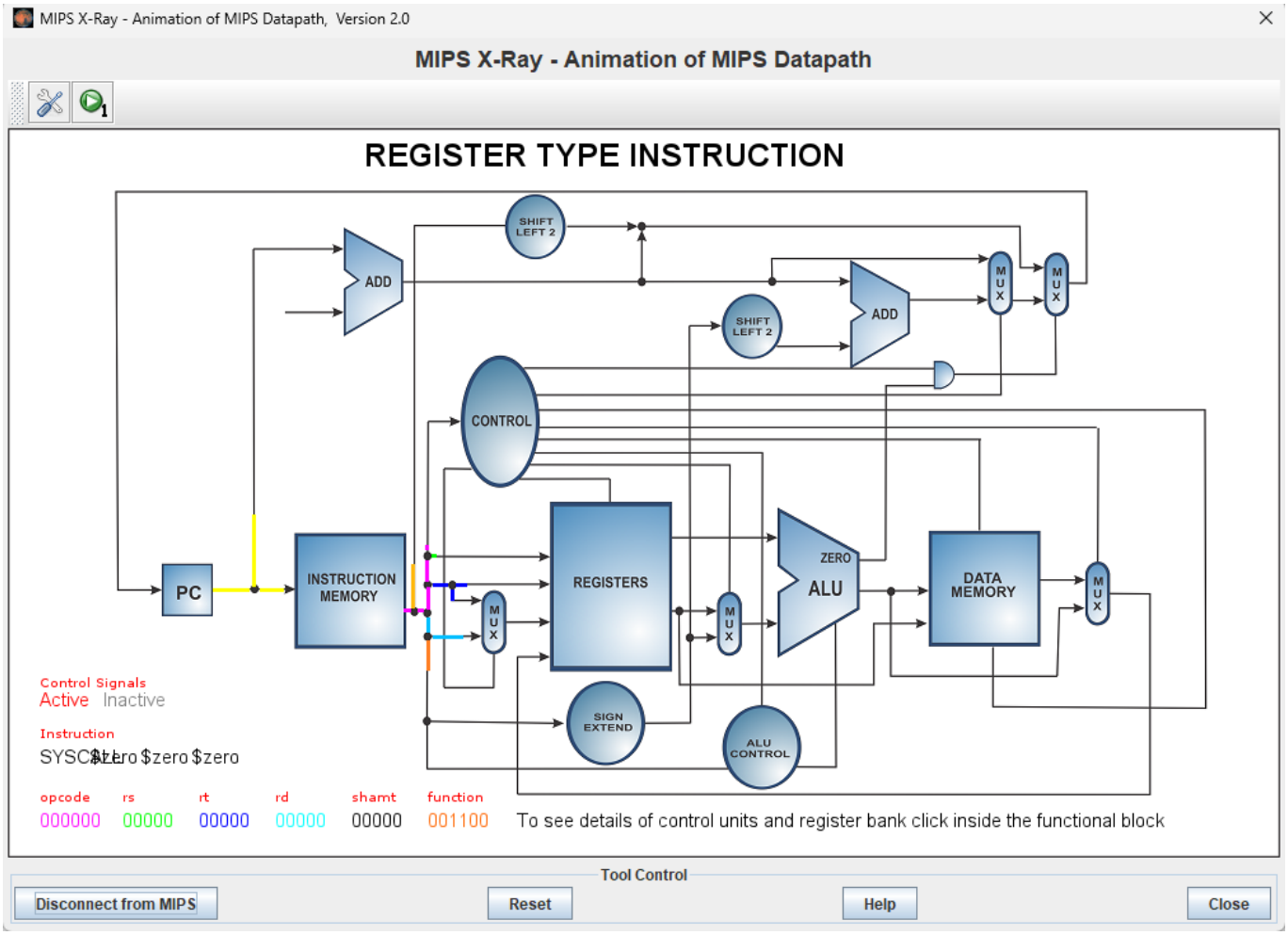
Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	95	94
Stalls (Paradas) por iteración	4 (2 de Load-Use y 2 del mul-addu)	0
Total de Stalls (8 iteraciones)	32 Stalls Totales	0
Ciclos Totales Estimados (Inst + Stalls)	95+32=127	94
CPI Estimado (Ciclos / Inst)	127/95≈1.34	1

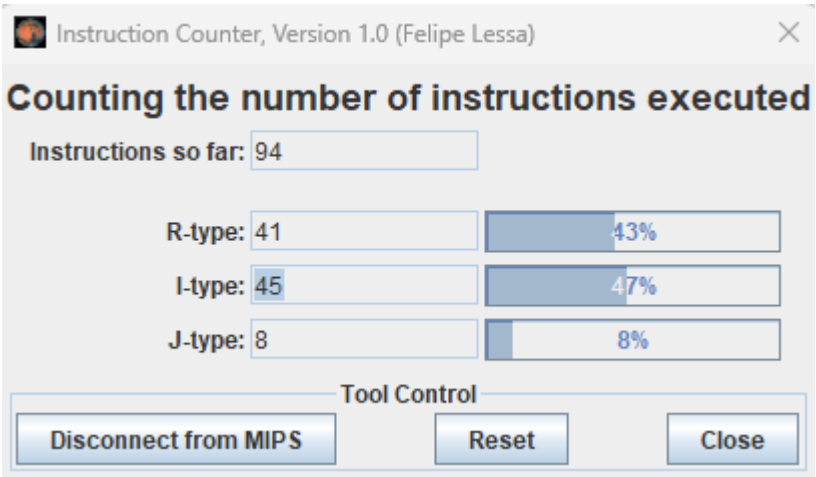
2. Optimización Propuesta

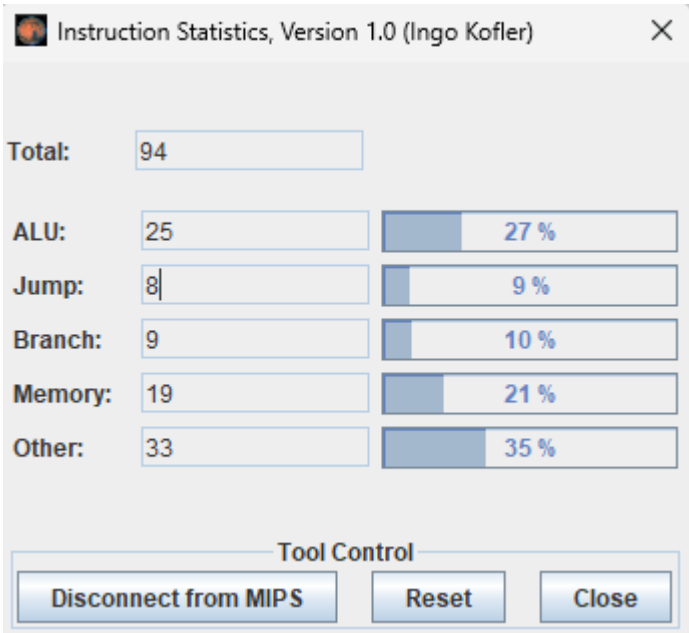
2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**.
- **Instruction Counter**.
- **Instruction Statistics**.


MIPS X-Ray - Animation of MIPS Datapath, Version 2.0
MIPS X-Ray - Animation of MIPS Datapath
REGISTER TYPE INSTRUCTION
SHIFT LEFT 2
ADD
ADD
SHIFT LEFT 2
MUX
MUX
CONTROL
PC
INSTRUCTION MEMORY
MUX
REGISTERS
MUX
ZERO
ALU
DATA MEMORY
MUX
SIGN EXTEND
ALU CONTROL
Control Signals
Active Inactive
Instruction
SYSCALL $zero $zero $zero
opcode rs rt rd shamt function
000000 00000 00000 00000 00000 001100
To see details of control units and register bank click inside the functional block
Tool Control
Disconnect from MIPS
Reset
Help
Close


Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 94
R-type: 41
43%
I-type: 45
47%
J-type: 8
8%
Tool Control
Disconnect from MIPS
Reset
Close



2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin     # Si i == tamano, salir del bucle

    # --- Cálculo de dirección de memoria ---
    sll $t4, $t3, 2       # Desplazamiento: t4 = i * 4
    addu $t5, $s0, $t4    # t5 = dirección de X[i]

    # --- Carga de dato ---
    lw $t6, 0($t5)        # Leer X[i]

    # Optimizacion No. 1: se mueve addu $t9 aquí para eliminar el stall Load-Use
(lw - mul)
    addu $t9, $s1, $t4    # t9 = dirección de Y[i]

    # --- Operación aritmética ---
    mul $t7, $t6, $t0     # t7 = X[i] * A  (sin stall: $t6 queda listo de una vez)

    # Optimizacion 2: se mueve addi aquí para eliminar el stall mul-addu (mul -
addu)
    addi $t3, $t3, 1      # i = i + 1

    addu $t8, $t7, $t1    # t8 = t7 + B (sin stall: $t7 mismo caso al anterior
queda listo de una)

    # --- Almacenamiento de resultado ---
    sw $t8, 0($t9)        # Guardar resultado en Y[i]

    j loop

fin:
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

Movi 2 instrucciones independientes para evitar los riesgos de datos del pipeline:

1. la primera optimizacion es al Load-Use(lw-mul): la instruccion "lw $t6, 0($t5)" necesita completar su etapa de memoria antes de que entre en juego "mul $t7, $t6, $t0" y pueda leer el valor "$t6" en el codigo original esto hacia que se hicieran 2 ciclos de parada, basicamente un trancon de datos. En este caso mi solucion fue mover "addu $t9, $s1, $t4" en medio de las instrucciones antes mencionadas ya que esta no depende de "$t6" ni tampoco va a generar valores que "mul" necesite para funcionar. De esta manera utiliza el hueco que queda en el piepeline, de una forma mucho mas util, ya que mientras que el "lw" le da un valor a "mul" en ese peuqeño pero importante lapso de timepo el addu se resuelve.

2. Optimizacion 2 es a la dependencia de RAW (nul - addu) En este caso la instruccion "mul $t7, $t6, $t0" tarda mas de 1 ciclo en producir un valor, esto hace que "addu $t8, $t7, $t1" se quede quieto esperando 2 ciclos para poder leer el "$t7". La solucion fue mover a "addi $t3, $t3, 1" en medio de esas instrucciones, basicamente la misma solucion que en el caso anterior. la instruccion que se deja en la mitad es totalmente independiente ya que no lee ni escribe a un registro que se involucre en el riesgo.

Como ya fue demostrado ambos cambios aunque simples, hacen la diferencia en el funcionamiento del codigo, ya que hace que el pipeline no se frene y siga avanzando, reduciendo el CPI de 1.34 a 1.00.

---

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	127	94	26%
Stalls (Paradas)	32	0	100%
CPI	1.34	1.00	25.4%

---

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas? Basicamente para resumirlo, busca hacer mas con menos o en algunos casos con un orden diferente. Busca la manera de volver cada uno de los procesos en algo mas efectivo, que aproveche cada momento que tenga en hacer diferentes tareas simultaneas pero que funcionan en armonia como un reloj, sin que ninguna estorbe o frene a las otras. Es importante por que hace que que los procesos sean mas rapidos y esto tiene el poder de repercutir en cada cosa que hace un ordenador, en este caso especifico hablamos de un "procesador", la segmentacion busca que el proceso sea casi instantaneo o lo mas rapido posible.

En un codigo simple como este creo que si es posible, en codigos mas complejos diria que la mayoria si se puede quitar, habran unas que tienes una solucion mucho mas compleja y unas pocas que no tendran una solucion debido a su complejidad.